

Как да напишем `course.cpp`

ръководство стъпка по стъпка

Microsoft Visual Studio · C++17 · `set`, `map`, шаблонни класове функции

Какво ще направим

Три вложени класа: студент, упражнение (с вектор от студенти) и дисциплина (с вектор от упражнения). Данните идват от текстов файл, а от тях се извличат три справки.

Справка	Тип на резултата	Защо този тип
всички студенти, подредени по фак. номер	<code>set<Student></code>	студентът се среща в няколко упражнения — множеството само маха повторенията и само подрежда
неповтарящи се групи за едно упражнение	<code>list<int></code>	условието иска списък
групи за всяко упражнение	<code>map<string, list<int>></code>	ключ — име на упражнение; стойността сама е контейнер

Новото спрямо упражнение 12. Там `map` беше справочник, който сами попълвахме. Тук `set` върши работа вместо нас: в упражнение 10 премахването на дубликати изискваше `sort + unique + erase`. С `set` това е един `insert` — повторните просто не влизат.

План на работата

1. Нов проект, скелет и входният файл
2. Класът `Student` (изискване I)
3. Класът `Exam` (изискване II)
4. Четене на ред с променлив брой полета
5. Класът `Course` и четенето на файла (изискване III)
6. Множеството от студенти — `set`
7. Списъкът от групи и картата `map<string, list<int>>`
8. Шаблонните класове `SetPrinter` и `ListPrinter` (изисквания IV–V)
9. Главна функция (изискване VI)

1 Нов проект, скелет и входният файл

Нов **Empty Project**, например `Tema13_Course`, файл `course.cpp`.

```
#include <algorithm> // sort, find
#include <cstdio>     // getchar
```

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <list>
#include <map>
#include <set>
#include <sstream>    // istringstream
#include <stdexcept> // runtime_error
#include <string>
#include <vector>
using namespace std;
```

В папката на проекта — `Тема13.txt` . Всяко упражнение е на **един ред**:

```
OOP
OOP1 106611 2 106621 3 106603 1 106601 1
OOP1KP 106621 3 106602 1 106631 4 106601 1 096042 5 096022 2 096033 3 096041 5 096002 1 096003
1 096011 2
OOP2 106611 2 106602 1 106603 1 096042 5
```

Вторият ред е дълъг и в PDF на условието се пренася. Във файла той трябва да е **един ред**, без прекъсване. Ако се разкъса на две, вторият къс ще бъде прочетен като ново упражнение с име `096042` .

Името на класа `Course` е така в условието — правилното английско е *Course*. Оставяме го както е зададено, за да съвпада с изискването. В реален проект такъв правопис се поправя веднага, докато класът се ползва само на едно място.

2 Класът `Student`

```
class Student
{
private:
    string m_strFN;    // факултетен номер
    int    m_nGroup;   // група

public:
    Student();          // подразбиращ се
    Student(const string& strFN, int nGroup);    // експлицитен

    const string& getFN() const;
    int getGroup() const;
    void setFN(const string& strFN);
    void setGroup(int nGroup);

    bool operator<(const Student& stud) const;

    friend ostream& operator<<(ostream& toStream, const Student& stud);
    friend istream& operator>>(istream& fromStream, Student& stud);
};
```

```
// От този оператор зависи и подредбата, и УНИКАЛНОСТТА в set:
// set смята за еднакви два елемента, при които нито a<b, нито b<a.
bool Student::operator<(const Student& stud) const
{
    return m_strFN < stud.m_strFN;
}
```

Прочетете коментара внимателно — това е ключът към цялото упражнение.

set няма operator== и не го използва. За да реши дали два елемента са един и същ, той пита **два пъти** оператора < :

```
if (!(a < b) && !(b < a)) → a и b са ЕДИН И СЪЩ елемент
```

Понеже нашият < гледа само факултетния номер, двама студенти с еднакъв номер и **различни групи** се смятат за един и същ студент. Вторият изобщо не влиза в множеството.

Точно това искаме тук — един студент е един студент, независимо колко пъти се среща. Но следствието трябва да е съзнателно: **какво сложите в operator<, това става определение за уникалност**. Ако бяхте сравнявали по номер и група, един и същ студент, записан веднъж в група 1 и веднъж в група 2, щеше да влезе два пъти.

3 Класът Exam

```
class Exam
{
private:
    string          m_strName;          // име на упражнението
    vector<Student> m_vecStudents;      // студентите, с които се провежда

public:
    Exam();
    explicit Exam(const string& strName);    // само по име на упражнението

    const string& getName() const;
    void setName(const string& strName);
    const vector<Student>& getStudents() const;

    void addStudent(const Student& stud);
    void sortStudents();                    // по факултетен номер

    bool operator<(const Exam& exam) const;    // по име на упражнение
    bool operator==(const Exam& exam) const;    // по име на упражнение

    friend ostream& operator<<(ostream& toStream, const Exam& exam);
    friend istream& operator>>(istream& fromStream, Exam& exam);
};
```

```
void Exam::sortStudents()
{
```

```
sort(m_vecStudents.begin(), m_vecStudents.end()); // operator< на Student
}
```

Един ред върши цялото сортиране. `std::sort` работи тук, защото контейнерът е `vector` (за `list` щеше да е `lst.sort()` — упражнение 11), а критерият е `operator<` на `Student` — същият, който определя уникалността в множеството. Написан веднъж, използван на три места.

4 Четене на ред с променлив брой полета

Тук е единственото истински ново нещо в четенето. Досега всички формати имаха фиксиран брой полета. Сега всеки ред има име на упражнение и **неизвестен брой** двойки „номер група“.

Наивното решение не работи:

```
fromStream >> exam.m_strName;
Student stud;
while (fromStream >> stud)           // НЕ правете така
    exam.m_vecStudents.push_back(stud);
```

`>>` не различава интервал от нов ред (упражнение 5). Затова цикълът ще продължи и на следващия ред, ще прочете `00P1KP` като факултетен номер, ще се опита да прочете `106621` като група и ще натъпче цялата дисциплина в едно упражнение.

РЕШЕНИЕТО: ПЪРВО РЕД, ПОСЛЕ РАЗЛАГАНЕ

```
// Форматът е <име> <фн> <група> <фн> <група>... до края на РЕДА,
// затова тук се чете ред и се разлага с istringstream.
istream& operator>>(istream& fromStream, Exam& exam)
{
    string strLine;
    // Прескачаме евентуални празни редове.
    while (getline(fromStream, strLine))
        if (!strLine.empty() && strLine.find_first_not_of(" \t\r") != string::npos)
            break;

    if (strLine.empty() || strLine.find_first_not_of(" \t\r") == string::npos)
        return fromStream;

    istringstream iss(strLine);
    if (!(iss >> exam.m_strName))
        return fromStream;

    exam.m_vecStudents.clear();
    Student stud;
    while (iss >> stud)           // до края на РЕДА, не на файла
        exam.m_vecStudents.push_back(stud);

    return fromStream;
}
```

- 1 `getline` чете **точно един ред**, заедно с интервалите, и спира на новия ред. Само тя знае къде свършва редът.
- 2 `istringstream iss(strLine)` прави поток **от този един ред**. Всичко останало е недостъпно за него.
- 3 `while (iss >> stud)` чете двойки, докато редът свърши. Броят им няма значение — цикълът спира сам.
- 4 `find_first_not_of(" \t\r")` проверява дали редът е само празни знаци. Връща `string::npos`, ако е.

Този похват — `getline` плюс `istringstream` — е стандартният отговор на всеки формат, при който редът има значение. Запомнете го: той решава и случая с улица, съдържаща интервали, от упражнение 5.

ЗНАКЪТ `\R` ПРИ ФАЙЛОВЕ ОТ WINDOWS

```
// Махаме евентуален '\r' от файл, записан под Windows.
while (!course.m_strName.empty() &&
      (course.m_strName.back() == '\r' || course.m_strName.back() == ' '))
    course.m_strName.erase(course.m_strName.size() - 1);
```

Под Windows края на ред е два знака — `\r\n`. `getline` маха само `\n` и оставя `\r` залепен за последната дума. Резултатът е име `"00P\r"` вместо `"00P"` — изглежда еднакво при извеждане, но `==` дава лъжа и търсенето по това име не намира нищо.

Ако отваряте файла в текстов режим под Windows, преобразуването става автоматично. Но файл, копиран от друга система, или четен в двоичен режим, носи `\r`. Двата реда по-горе струват нищо и спестяват час търсене.

5 Класът `Course` и четенето на файла

```
class Course
{
private:
    string      m_strName;          // име на дисциплината
    vector<Exam> m_vecExams;        // упражнения, които включва

public:
    Course();
    explicit Course(const string& strFileName);    // само по име на файл

    const string& getName() const;
    void setName(const string& strName);
    const vector<Exam>& getExams() const;

    set<Student> getStudentSet() const;
    list<int> getGroups(const string& strExamName) const;
    map<string, list<int>> getGroupsMap() const;

    friend ostream& operator<<(ostream& toStream, const Course& course);
    friend istream& operator>>(istream& fromStream, Course& course);
};
```

Интервалът в `map<string, list<int> >` не е случаен. При по-стари компилатори `>>` се разчита като оператора за четене и се получава грешка C2947: `expecting '>' to terminate template-argument-list`. От C++11 нататък интервалът не е нужен, но не пречи и е добър навик при вложени шаблони.

```
istream& operator>>(istream& fromStream, Course& course)
{
    // Първият ред е името на дисциплината.
    if (!getline(fromStream, course.m_strName))
        return fromStream;

    ... (изчистване на '\r')

    course.m_vecExams.clear();
    Exam exam;
    while (fromStream >> exam)
        course.m_vecExams.push_back(exam);

    return fromStream;
}
```

Конструкторът по име на файл е като в упражнения 10–12: отваря, чете чрез оператора, хвърля `runtime_error` при неуспех.

6 Множеството от студенти

```
// set сам подрежда по operator< и сам премахва повторенията.
set<Student> Course::getStudentSet() const
{
    set<Student> setStudents;
    for (size_t i = 0; i < m_vecExams.size(); i++)
    {
        const vector<Student>& vec = m_vecExams[i].getStudents();
        for (size_t j = 0; j < vec.size(); j++)
            setStudents.insert(vec[j]);    // повторните просто не влизат
    }
    return setStudents;
}
```

Няма нито една проверка за дубликат и нито едно сортиране. `insert` сам намира мястото по `operator<` и сам отказва, ако елементът вече го има. В примера 19 записа дават 13 различни студенти.

Сравнете с упражнение 10, където същото искаше `sort`, после `unique`, после `erase` — и всяка от трите стъпка можеше да се пропусне.

ТЪРСЕНЕ В МНОЖЕСТВОТО

```
Student target(arrFind[i], 0);    // групата няма значение за <
// find на set търси по operator<, не с обхождане – затова е бързо.
set<Student>::const_iterator it = setStud.find(target);
```

`setStud.find(...)` и `std::find(setStud.begin(), ...)` са две различни неща.

Функцията на контейнера използва вътрешната подредба и стига до елемента за около $\log_2 n$ сравнения. **Глобалният алгоритъм** обхожда от началото — n сравнения — и при това ползва `operator==`, какъвто `Student` дори няма.

Правилото: ако контейнерът има собствена функция със същото име, тя винаги е по-добрата. Същото важи за `count` при `map` и `set`, и за `sort` при `list`.

7 Списъкът от групи и картата

```
list<int> Course::getGroups(const string& strExamName) const
{
    list<int> lstGroups;

    for (size_t i = 0; i < m_vecExams.size(); i++)
    {
        if (m_vecExams[i].getName() != strExamName)
            continue;

        const vector<Student>& vec = m_vecExams[i].getStudents();
        for (size_t j = 0; j < vec.size(); j++)
        {
            int nGroup = vec[j].getGroup();
            // Добавяме само ако групата още я няма в списъка.
            if (find(lstGroups.begin(), lstGroups.end(), nGroup) == lstGroups.end())
                lstGroups.push_back(nGroup);
        }
    }

    lstGroups.sort();           // собствената функция на list
    return lstGroups;
}
```

Тук уникалността се пази на ръка, защото условието иска `list<int>`, а списъкът не прави нищо сам. Проверката с `find` преди всяко добавяне е точно това, което `set` върши вместо нас в стъпка 6.

Ако резултатът можеше да е `set<int>`, цялото тяло щеше да е един `insert`. Разликата между двете функции в този файл показва какво точно ви дава изборът на контейнер.

`lstGroups.sort()`, не `sort(lstGroups.begin(), lstGroups.end())` — `list` няма произволен достъп (упражнение 11).

КАРТАТА, ЧИЯТО СТОЙНОСТ Е КОНТЕЙНЕР

```
map<string, list<int> > Course::getGroupsMap() const
{
    map<string, list<int> > mapResult;
    for (size_t i = 0; i < m_vecExams.size(); i++)
    {
        const string& strName = m_vecExams[i].getName();
        mapResult[strName] = getGroups(strName);
    }
}
```

```

    }
    return mapResult;
}

```

Стойността в картата може да бъде какъвто и да е тип — включително цял контейнер. `map<string, list<int>>` е „за всяко упражнение — списък от групи“, което е точно каквото иска условието.

Тук `operator[]` се използва **умишлено**: при ключ, който още го няма, той създава запис с празен списък и връща него — а ние веднага му присвояваме резултата. Точно затова в упражнение 12 предупредихме да не се ползва `[]` за **четене**: при запис това поведение е желано, при четене — почти никога.

8 Шаблонните класове функции

```

template <class T>
class SetPrinter
{
public:
    void operator()(const set<T>& setData) const
    {
        cout << " множество (" << setData.size() << "):";
        for (typename set<T>::const_iterator it = setData.begin();
             it != setData.end(); ++it)
            cout << " " << *it;
        cout << endl;
    }
};

template <class T>
class ListPrinter
{
public:
    void operator()(const string& strName, const list<T>& lstData) const
    {
        cout << " " << strName << " (" << lstData.size() << "):";
        for (typename list<T>::const_iterator it = lstData.begin();
             it != lstData.end(); ++it)
            cout << " " << *it;
        cout << endl;
    }
};

```

ДУМАТА `TYPENAME`

`typename set<T>::const_iterator` — без `typename` не се компилира.

Причината: докато `T` е неизвестно, компилаторът не знае дали `set<T>::const_iterator` е **тип** или **статична променлива**. По подразбиране приема второто и дава `C2825: 'set<T>': must be a class or namespace when followed by '::'`. Думата `typename` му казва, че е тип.

Правилото: **вътре в шаблон, пред всяко име от вида `ИмеНаТип<T>::Нещо`, се пише `typename`**. Извън шаблон не е нужно — там типът е известен.

Това са функционални обекти (функтори), като CLess от упражнение 6: класове с operator() , които се викат като функции:

```
SetPrinter<Student> printSet;  
printSet(setStud);           // вика printSet.operator()(setStud)
```

Условието ги иска като класове, а не като обикновени шаблонни функции. Практическата разлика е, че функторът може да носи състояние — например ширина на колоната или брой елементи на ред, зададени в конструктора.

9 Главна функция

9.1 Създаване и извеждане

```
Course course;  
try  
{  
    course = Course("Тема13.txt");  
    cout << " прочетена дисциплина: " << course.getName()  
        << ", упражнения: " << course.getExams().size() << endl;  
}  
catch (const exception& ex)  
{  
    cout << " хванато изключение: " << ex.what() << endl;  
    getchar();  
    return 1;  
}  
  
cout << course;
```

9.2 Проверка, че множеството наистина маха повторенията

```
size_t nTotal = 0;  
for (size_t i = 0; i < course.getExams().size(); i++)  
    nTotal += course.getExams()[i].getStudents().size();  
  
set<Student> setStud = course.getStudentSet();  
cout << " общо записа във всички упражнения: " << nTotal << endl;  
cout << " различни студенти в множеството: " << setStud.size() << endl;  
  
SetPrinter<Student> printSet;           // IV. шаблонният клас функция  
printSet(setStud);
```

19 записа → 13 студента. Изведеното множество е подредено по факултетен номер, макар никъде да не е викано сортиране.

9.3 Групите по упражнения

```
ListPrinter<int> printList;           // V. шаблонният клас функция  
  
for (size_t i = 0; i < vecExams.size(); i++)  
{
```

```

    const string& strName = vecExams[i].getName();
    printList("групи за " + strName, course.getGroups(strName));
}

printList("групи за НЕСъществуващо упражнение", course.getGroups("НЯМА"));

```

Последният ред проверява граничния случай: несъществуващо име дава празен списък, а не грешка.

9.4 Шаблоните върху други типове

```

set<string> setNames;
setNames.insert("Петров");
setNames.insert("Иванов");
setNames.insert("Георгиев");
setNames.insert("Иванов"); // повторение – не влиза

SetPrinter<string> printSetStr;
printSetStr(setNames);

```

Същите два класа работят за **Student**, **string** и **int**, без нито ред промяна. Единственото изискване към типа е да има `operator<<` (за извеждането) и `operator<` (за да може да влезе в `set`). И трите типа ги имат — за `Student` защото ги написахме в стъпка 2.

От четирите вмъкнати имена излизат три: второто „Иванов“ не влиза. И тук уникалността е по `operator<` — този на `string`.

Компилиране и стартиране

Клавиши	Действие
<code>Ctrl</code> + <code>Shift</code> + <code>B</code>	Build.
<code>F5</code>	Start Debugging. Работната папка е тази на проекта — оттам се търси <code>Тема13.txt</code> .
<code>F9</code>	Точка на прекъсване в <code>operator>></code> на <code>Exam</code> . В Watch следете <code>strLine</code> — вижда се, че всеки ред се обработва отделно.

```

cl /EHsc /std:c++17 /utf-8 course.cpp
course.exe

```

Кирилица под Windows

Файлът се записва като *Unicode (UTF-8 with signature)* — *Codepage 65001*; компилаторът получава `/utf-8`; конзолата се превключва с `chcp 65001` или със `system("chcp 65001 > nul");` като първи ред в `main`.

Данните в `Тема13.txt` са на латиница и цифри, така че четенето не зависи от кодировката. Кирилица има само в съобщенията на програмата. Забележете и че `set<string>` с кирилски имена се поддържа по байтове на UTF-8 — за българската азбука това съвпада с азбучния ред, но за език с диакритични знаци не би съвпаднало.

Пълен код на програмата

Ако някъде сте се загубили, сравнете с този листинг:

```
// Тема 13: Асоциативни контейнери, алгоритми.
// Student – фак. номер и група.
// Exam – упражнение с вектор от студенти.
// Course – дисциплина с вектор от упражнения. (Името е както в условието.)

#include <algorithm> // sort, find
#include <cstdio>    // getchar
#include <fstream>
#include <iomanip>
#include <iostream>
#include <list>
#include <map>
#include <set>
#include <sstream>    // stringstream
#include <stdexcept> // runtime_error
#include <string>
#include <vector>
using namespace std;

// =====
// I. Клас Student
// =====
class Student
{
// Скрити членове
private:
    string m_strFN;    // факултетен номер
    int    m_nGroup;   // група

public:
    Student();          // подразбиращ се
    Student(const string& strFN, int nGroup); // експлицитен

    // Четене / промяна на членовете
    const string& getFN() const;
    int getGroup() const;
    void setFN(const string& strFN);
    void setGroup(int nGroup);

    // Сравнение за по-малък по факултетен номер
    bool operator<(const Student& stud) const;

    // Поток в вход/изход
    friend ostream& operator<<(ostream& toStream, const Student& stud);
    friend istream& operator>>(istream& fromStream, Student& stud);
};

Student::Student() : m_strFN("(няма)"), m_nGroup(0) {}

Student::Student(const string& strFN, int nGroup)
    : m_strFN(strFN), m_nGroup(nGroup) {}
```

```

const string& Student::getFN() const { return m_strFN; }
int Student::getGroup() const { return m_nGroup; }
void Student::setFN(const string& strFN) { m_strFN = strFN; }
void Student::setGroup(int nGroup) { m_nGroup = nGroup; }

// ----- Подредба по факултетен номер -----
// От този оператор зависи и подредбата, и УНИКАЛНОСТТА в set:
// set смята за еднакви два елемента, при които нито a<b, нито b<a.
bool Student::operator<(const Student& stud) const
{
    return m_strFN < stud.m_strFN;
}

ostream& operator<<(ostream& toStream, const Student& stud)
{
    toStream << stud.m_strFN << "/" << stud.m_nGroup;
    return toStream;
}

istream& operator>>(istream& fromStream, Student& stud)
{
    string strFN;
    int nGroup = 0;
    if (fromStream >> strFN >> nGroup)
    {
        stud.m_strFN = strFN;
        stud.m_nGroup = nGroup;
    }
    return fromStream;
}

// =====
// II. Клас Exam
// =====
class Exam
{
private:
    string m_strName; // име на упражнението
    vector<Student> m_vecStudents; // студентите, с които се провежда

public:
    Exam(); // подразбира се
    explicit Exam(const string& strName); // само по име на упражнението

    const string& getName() const;
    void setName(const string& strName);
    const vector<Student>& getStudents() const;

    void addStudent(const Student& stud);
    void sortStudents(); // по факултетен номер

    bool operator<(const Exam& exam) const; // по име на упражнение
    bool operator==(const Exam& exam) const; // по име на упражнение

    friend ostream& operator<<(ostream& toStream, const Exam& exam);
    friend istream& operator>>(istream& fromStream, Exam& exam);
};

Exam::Exam() : m_strName("(няма)"), m_vecStudents() {}

```

```

Exam::Exam(const string& strName) : m_strName(strName), m_vecStudents() {}

const string& Exam::getName() const { return m_strName; }
void Exam::setName(const string& strName) { m_strName = strName; }
const vector<Student>& Exam::getStudents() const { return m_vecStudents; }

void Exam::addStudent(const Student& stud) { m_vecStudents.push_back(stud); }

// ----- Сортиране на студентите по фак. номер -----
void Exam::sortStudents()
{
    sort(m_vecStudents.begin(), m_vecStudents.end()); // operator< на Student
}

bool Exam::operator<(const Exam& exam) const { return m_strName < exam.m_strName; }
bool Exam::operator==(const Exam& exam) const { return m_strName == exam.m_strName; }

ostream& operator<<(ostream& toStream, const Exam& exam)
{
    toStream << exam.m_strName << " (" << exam.m_vecStudents.size() << "):";
    for (size_t i = 0; i < exam.m_vecStudents.size(); i++)
        toStream << " " << exam.m_vecStudents[i];
    return toStream;
}

// ----- Четене на ЕДИН ред от файла -----
// Форматът е <име> <фн> <група> <фн> <група>... до края на РЕДА,
// затова тук се чете ред и се разлага с istringstream.
istream& operator>>(istream& fromStream, Exam& exam)
{
    string strLine;
    // Прескачае евентуални празни редове.
    while (getline(fromStream, strLine))
        if (!strLine.empty() && strLine.find_first_not_of(" \t\r") != string::npos)
            break;

    if (strLine.empty() || strLine.find_first_not_of(" \t\r") == string::npos)
        return fromStream;

    istringstream iss(strLine);
    if (!(iss >> exam.m_strName))
        return fromStream;

    exam.m_vecStudents.clear();
    Student stud;
    while (iss >> stud) // до края на РЕДА, не на файла
        exam.m_vecStudents.push_back(stud);

    return fromStream;
}

// =====
// III. Клас Course (дисциплина)
// =====
class Course
{
private:
    string m_strName; // име на дисциплината

```

```

vector<Exam> m_vecExams;    // упражнения, които включва

public:
    Course();                // подразбира се
    explicit Course(const string& strFileName);    // само по име на файл

    const string& getName() const;
    void setName(const string& strName);
    const vector<Exam>& getExams() const;

    // Множество от студентите, подредени по фак. номер
    set<Student> getStudentSet() const;

    // Списък от неповтарящи се групи за ЗАДАДЕНО упражнение
    list<int> getGroups(const string& strExamName) const;

    // Карта: име на упражнение -> списък от неповтарящи се групи
    map<string, list<int> > getGroupsMap() const;

    friend ostream& operator<<(ostream& toStream, const Course& course);
    friend istream& operator>>(istream& fromStream, Course& course);
};

Course::Course() : m_strName("(няма)"), m_vecExams() {}

Course::Course(const string& strFileName) : m_strName("(няма)"), m_vecExams()
{
    ifstream iFile(strFileName.c_str());
    if (!iFile)
        throw runtime_error("Не мога да отворя файла: " + strFileName);

    iFile >> *this;

    if (m_vecExams.empty())
        throw runtime_error("Файлт " + strFileName + " не съдържа упражнения");
}

const string& Course::getName() const    { return m_strName; }
void Course::setName(const string& strName)    { m_strName = strName; }
const vector<Exam>& Course::getExams() const    { return m_vecExams; }

// ----- Четене на цялата дисциплина -----
istream& operator>>(istream& fromStream, Course& course)
{
    // Първият ред е името на дисциплината.
    if (!getline(fromStream, course.m_strName))
        return fromStream;

    // Махаме евентуален '\r' от файл, записан под Windows.
    while (!course.m_strName.empty() &&
           (course.m_strName.back() == '\r' || course.m_strName.back() == ' '))
        course.m_strName.erase(course.m_strName.size() - 1);

    course.m_vecExams.clear();
    Exam exam;
    while (fromStream >> exam)
        course.m_vecExams.push_back(exam);

    return fromStream;
}

```

```

}

ostream& operator<<(ostream& toStream, const Course& course)
{
    toStream << "Дисциплина " << course.m_strName
        << " (" << course.m_vecExams.size() << " упражнения)" << endl;
    for (size_t i = 0; i < course.m_vecExams.size(); i++)
        toStream << "    " << course.m_vecExams[i] << endl;
    return toStream;
}

// ----- Множество от студентите -----
// set сам подрежда по operator< и сам премахва повторенията.
set<Student> Course::getStudentSet() const
{
    set<Student> setStudents;
    for (size_t i = 0; i < m_vecExams.size(); i++)
    {
        const vector<Student>& vec = m_vecExams[i].getStudents();
        for (size_t j = 0; j < vec.size(); j++)
            setStudents.insert(vec[j]);    // повторните просто не влизат
    }
    return setStudents;
}

// ----- Неповтарящи се групи за дадено упражнение -----
list<int> Course::getGroups(const string& strExamName) const
{
    list<int> lstGroups;

    for (size_t i = 0; i < m_vecExams.size(); i++)
    {
        if (m_vecExams[i].getName() != strExamName)
            continue;

        const vector<Student>& vec = m_vecExams[i].getStudents();
        for (size_t j = 0; j < vec.size(); j++)
        {
            int nGroup = vec[j].getGroup();
            // Добавяме само ако групата още я няма в списъка.
            if (find(lstGroups.begin(), lstGroups.end(), nGroup) == lstGroups.end())
                lstGroups.push_back(nGroup);
        }
    }

    lstGroups.sort();    // собствената функция на list
    return lstGroups;
}

// ----- Карта: упражнение -> групи -----
map<string, list<int> > Course::getGroupsMap() const
{
    map<string, list<int> > mapResult;
    for (size_t i = 0; i < m_vecExams.size(); i++)
    {
        const string& strName = m_vecExams[i].getName();
        mapResult[strName] = getGroups(strName);
    }
    return mapResult;
}

```

```

}

// =====
// IV. Шаблонен клас функция SetPrinter
// =====
template <class T>
class SetPrinter
{
public:
    void operator()(const set<T>& setData) const
    {
        cout << " множество (" << setData.size() << "):";
        for (typename set<T>::const_iterator it = setData.begin();
             it != setData.end(); ++it)
            cout << " " << *it;
        cout << endl;
    }
};

// =====
// V. Шаблонен клас функция ListPrinter
// =====
template <class T>
class ListPrinter
{
public:
    void operator()(const string& strName, const list<T>& lstData) const
    {
        cout << " " << strName << " (" << lstData.size() << "):";
        for (typename list<T>::const_iterator it = lstData.begin();
             it != lstData.end(); ++it)
            cout << " " << *it;
        cout << endl;
    }
};

// =====
// VI. Главна функция
// =====
int main()
{
    // ===== Създаване на дисциплина по файл =====
    cout << "===== Създаване на Course по файл =====" << endl;

    try
    {
        Course bad("няма-такъв-файл.txt");
        cout << bad;
    }
    catch (const exception& ex)
    {
        cout << " хванато изключение: " << ex.what() << endl;
    }

    Course course;
    try
    {
        course = Course("Тема13.txt");
        cout << " прочетена дисциплина: " << course.getName()

```

```

        << " , упражнения: " << course.getExams().size() << endl;
    }
    catch (const exception& ex)
    {
        cout << " хванато изключение: " << ex.what() << endl;
        getchar();
        return 1;
    }

    // ===== Извеждане =====
    cout << endl << "===== Извеждане (operator<<) =====" << endl;
    cout << course;

    // ===== Сортиране на студентите в упражнение =====
    cout << endl << "===== Exam::sortStudents() =====" << endl;
    Exam exam1 = course.getExams()[0];
    cout << " преди: " << exam1 << endl;
    exam1.sortStudents();
    cout << " след: " << exam1 << endl;

    // ===== Множество от студентите =====
    cout << endl << "===== set<Student> - подредени и без повторения =====" << endl;

    size_t nTotal = 0;
    for (size_t i = 0; i < course.getExams().size(); i++)
        nTotal += course.getExams()[i].getStudents().size();

    set<Student> setStud = course.getStudentSet();
    cout << " общо записа във всички упражнения: " << nTotal << endl;
    cout << " различни студенти в множеството: " << setStud.size() << endl;

    SetPrinter<Student> printSet; // IV. шаблонният клас функция
    printSet(setStud);

    // Търсене в множеството
    cout << "---- Търсене в множеството ----" << endl;
    const char* arrFind[] = { "106601", "096042", "999999" };
    for (size_t i = 0; i < sizeof(arrFind) / sizeof(arrFind[0]); i++)
    {
        Student target(arrFind[i], 0); // групата няма значение за <
        // find на set търси по operator<, не с обхождане – затова е бързо.
        set<Student>::const_iterator it = setStud.find(target);
        cout << " " << arrFind[i] << " -> ";
        if (it != setStud.end())
            cout << "намерен: " << *it << endl;
        else
            cout << "НЕ е намерен" << endl;
    }

    // ===== Неповтарящи се групи по упражнение =====
    cout << endl << "===== list<int> – групи за дадено упражнение =====" << endl;
    ListPrinter<int> printList; // V. шаблонният клас функция

    const vector<Exam>& vecExams = course.getExams();
    for (size_t i = 0; i < vecExams.size(); i++)
    {
        const string& strName = vecExams[i].getName();
        list<int> lstGroups = course.getGroups(strName);
        printList("групи за " + strName, lstGroups);
    }

```

```

}

printList("групи за НЕсъществуващо упражнение", source.getGroups("НЯМА"));

// ===== Картата упражнение -> групи =====
cout << endl << "===== map<string, list<int>> =====" << endl;
map<string, list<int> > mapGroups = source.getGroupsMap();

cout << " размер на картата: " << mapGroups.size() << endl;
for (map<string, list<int> >::const_iterator it = mapGroups.begin();
     it != mapGroups.end(); ++it)
{
    cout << " " << setw(8) << left << it->first << "->";
    for (list<int>::const_iterator itg = it->second.begin();
         itg != it->second.end(); ++itg)
        cout << " " << *itg;
    cout << endl;
}

// Достъп по ключ
cout << "---- Достъп по ключ ----" << endl;
map<string, list<int> >::const_iterator itFound = mapGroups.find("00P1KP");
if (itFound != mapGroups.end())
    printList("00P1KP", itFound->second);

cout << " count(\"00P1\") = " << mapGroups.count("00P1") << endl;
cout << " count(\"НЯМА\") = " << mapGroups.count("НЯМА") << endl;

// ===== Шаблоните работят и с други типове =====
cout << endl << "===== Шаблоните са независими от типа =====" << endl;

set<string> setNames;
setNames.insert("Петров");
setNames.insert("Иванов");
setNames.insert("Георгиев");
setNames.insert("Иванов"); // повторение – не влиза

SetPrinter<string> printSetStr;
printSetStr(setNames);

set<int> setNums;
setNums.insert(42);
setNums.insert(7);
setNums.insert(42);
setNums.insert(13);

SetPrinter<int> printSetInt;
printSetInt(setNums);

list<string> lstWords;
lstWords.push_back("едно");
lstWords.push_back("две");
lstWords.push_back("три");

ListPrinter<string> printListStr;
printListStr("думи", lstWords);

// Задържа конзолния прозорец отворен при стартиране с F5 (debug)
getchar();

```

```
    return 0;
}
```

Очакван изход

```
===== Създаване на Course по файл =====
    хванато изключение: Не мога да отворя файла: няма-такъв-файл.txt
    прочетена дисциплина: OOP, упражнения: 3

===== Извеждане (operator<<) =====
Дисциплина OOP (3 упражнения)
    OOP1 (4): 106611/2 106621/3 106603/1 106601/1
    OOP1KP (11): 106621/3 106602/1 106631/4 106601/1 096042/5 096022/2 096033/3 096041/5
096002/1 096003/1 096011/2
    OOP2 (4): 106611/2 106602/1 106603/1 096042/5

===== Exam::sortStudents() =====
    преди: OOP1 (4): 106611/2 106621/3 106603/1 106601/1
    след:   OOP1 (4): 106601/1 106603/1 106611/2 106621/3

===== set<Student> - подредени и без повторения =====
    общо записа във всички упражнения: 19
    различни студенти в множеството:   13
    множество (13): 096002/1 096003/1 096011/2 096022/2 096033/3 096041/5 096042/5 106601/1
106602/1 106603/1 106611/2 106621/3 106631/4
--- Търсене в множеството ---
    106601 -> намерен: 106601/1
    096042 -> намерен: 096042/5
    999999 -> НЕ е намерен

===== list<int> - групи за дадено упражнение =====
    групи за OOP1 (3): 1 2 3
    групи за OOP1KP (5): 1 2 3 4 5
    групи за OOP2 (3): 1 2 5
    групи за НЕсъществуващо упражнение (0):

===== map<string, list<int>> =====
    размер на картата: 3
    OOP1    -> 1 2 3
    OOP1KP  -> 1 2 3 4 5
    OOP2    -> 1 2 5
--- Достъп по ключ ---
    OOP1KP (5): 1 2 3 4 5
    count("OOP1")    = 1
    count("НЯМА")    = 0

===== Шаблоните са независими от типа =====
    множество (3): Георгиев Иванов Петров
    множество (3): 7 13 42
    думи (3): едно две три
```

Чести грешки

Код	Съобщение	Причина и решение
-----	-----------	-------------------

C2825	'set<T>': must be a class or namespace when followed by '...'	Липсва <code>typename</code> пред <code>set<T>::const_iterator</code> в шаблон (стъпка 8).
C2947	expecting '>' to terminate template-argument-list	Нужен е интервал: <code>map<string, list<int> ></code> (стъпка 5).
C2676	binary '<': 'Student' does not define this operator	<code>set</code> изисква <code>operator<</code> , и то маркиран с <code>const</code> .
C3892	you cannot assign to a variable that is const	Опит за промяна на елемент на <code>set</code> . Елементите му са константни — промяната би развалила подредбата. Маха се и се вмъква наново.
LNK2019	unresolved external symbol ... SetPrinter<...>	Тялото на шаблона е отделено от обявяването. Шаблоните се пишат цели на едно място.
—	Прочетено е само 1 упражнение с много студенти	Използван е <code>>></code> вместо <code>getline</code> + <code>istringstream</code> (стъпка 4).
—	Прочетени са повече упражнения от очакваното	Дългият ред във файла е разкъсан на два (стъпка 1).
—	Търсене по име на упражнение не намира нищо	В името е останал <code>'\r'</code> от Windows-файл (стъпка 4).
—	Множеството съдържа повторени студенти	<code>operator<</code> сравнява и групата. Уникалността се определя от него (стъпка 2).
—	Множеството е по-малко от очакваното	Същата причина в обратна посока: <code>operator<</code> сравнява твърде малко.
—	Картата расте при търсене	<code>[]</code> при четене добавя празен запис. За четене — <code>find</code> (упражнение 12).

Обобщение: кой контейнер какво прави сам

Контейнер	Поддържа сам	Маха повторения сам	Търси бързо
<code>vector</code> , <code>list</code>	не	не	не
<code>set</code>	да — по <code>operator<</code>	да	да — <code>.find()</code>
<code>multiset</code>	да	не	да
<code>map</code>	да — по ключ	да — по ключ	да
<code>multimap</code>	да	не	да — <code>equal_range</code>

Изборът на контейнер е избор на това какво няма да пишете. Упражнение 10 премахваше дубликати с три стъпки; тук същото е един `insert`. Разликата не е в уменията, а в това дали контейнерът е подбран по задачата.

И в двата случая обаче цената е една и съща: правилният `operator<`. Той определя и подредбата, и уникалността — затова е първото, което се обмисля, и последното, което се променя.

Обратно към началото

В тема 1 пазехме име и пол в `char m_szName[64]` и внимавахме да не препълним буфера. В тема 3 управлявахме динамична памет на ръка с деструктор, копиращ конструктор и оператор `=`. Тук един `set<Student>` върши повече от всичко това и не може да се сгреша.

Пътят дотук не е бил заобиколен. Без тема 3 не е ясно защо копирането на контейнер не е безплатно; без темите 4 и 5 не е ясно защо `set` изобщо се нуждае от вашия `operator<`; без тема 6 — какво е шаблон и защо `SetPrinter<Student>` работи, без някой да го е писал за `Student`.